

ASTAR

HASH CRACKING : AUTOMATION DRIVEN BY LAZINESS, 10 YEARS AFTER

INTRODUCTION

We will mainly discuss about the passwords hashes you encounter during infrastructure pentest (aka internal pentest, aka corporate pentest, ...)

- Hashes caught through responder
- Hashes caught from SAM dump of a compromised box
- Hashes caught from SQL dump through SQLi
- ...

But the topic is about tactical cracking of hashes, in general.

The author : @Sibwara, pentester since 2011

BREAK HASHES

THE PASSWORD CRACKING BIAIS

During years, we used **bruteforce** to crack hashes.

Passwords became complex (at least «complexier»).

So we added **dictionnary attacks** based on **probable passwords**.

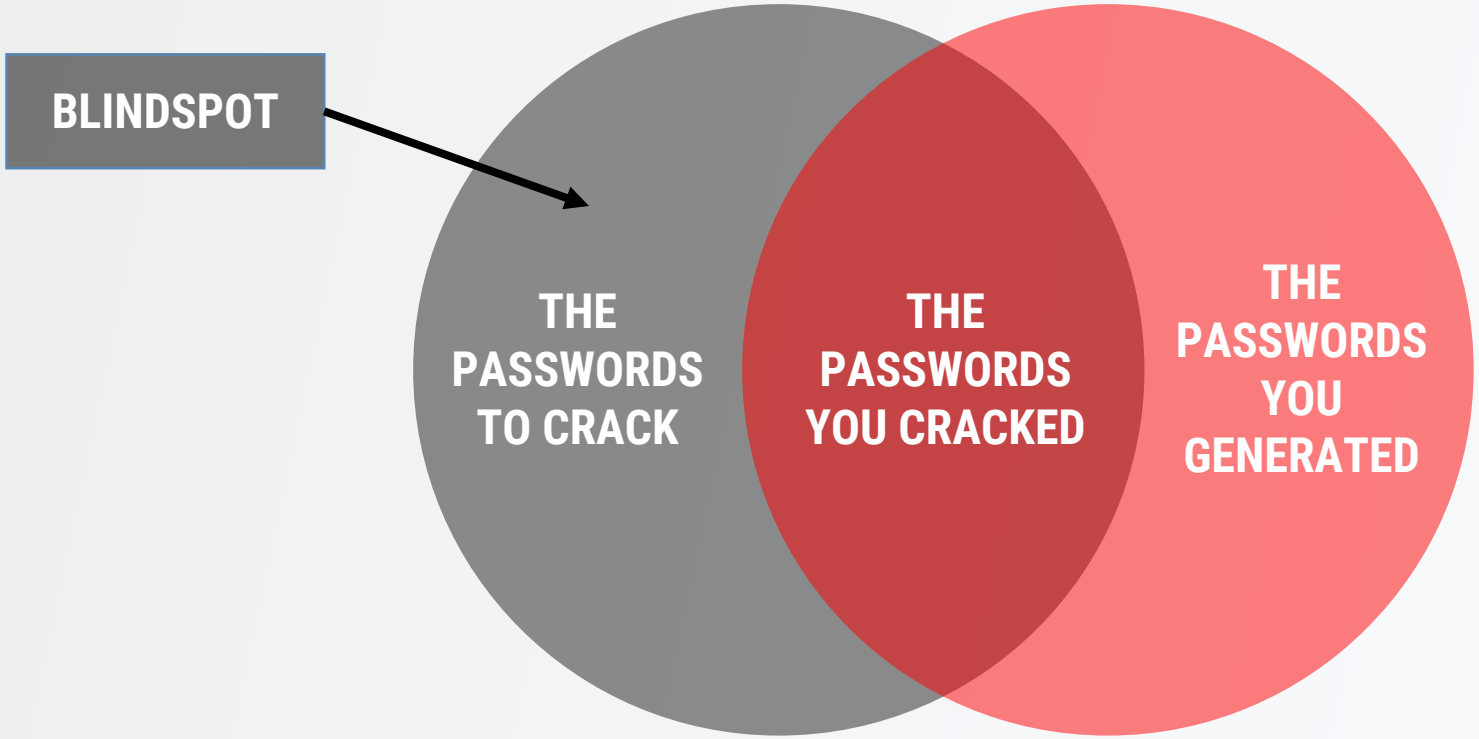
Then **dictionnary attacks** based on **predictable passwords**.

But we were half blind during those years !

THE PASSWORD CRACKING BIAIS

Indeed, the biais with predictable passwords attacks is that **you can only know when your guess is correct**.

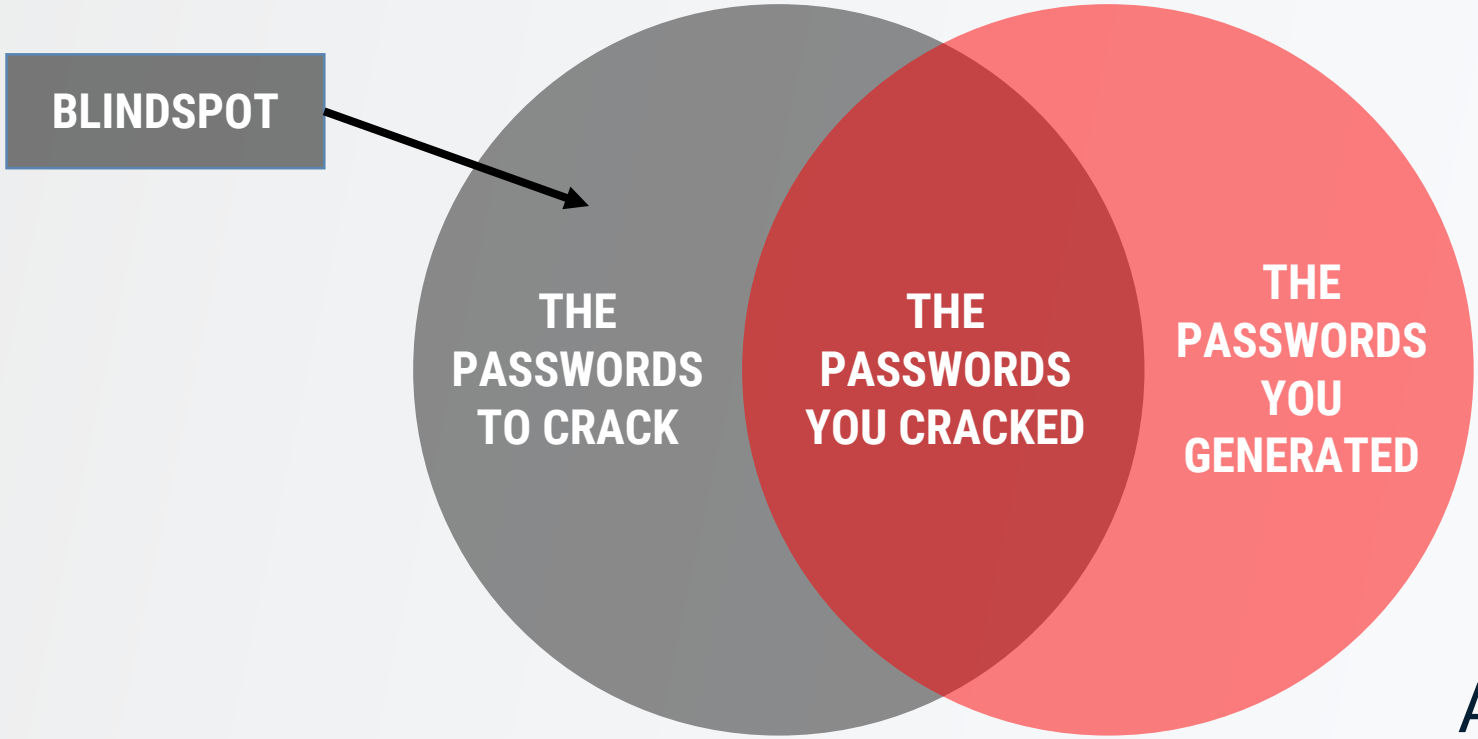
So you can only break passwords that you are able to imagine !



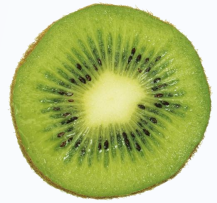
THE PASSWORD CRACKING BIAIS

Indeed, the biais with predictable passwords attacks is that **you can only know when your guess is correct**.

So you can only break passwords that you are able to imagine !



At least, until 2011



THE PASSWORD CRACKING BIAIS

When Mimikatz appeared, we were able to discover cleartext passwords without breaking the hashes.

So we opened our second eye !

The strange passwords, the complex passwords, the shameful passwords, the very unimaginable passwords ... were all naked.

Now, you can directly learn from observable users habits to tune your predictable passwords attacks !

So, I suggest always performing a widespread mimikatz over a network, even if you already compromised a domain admin (et least at the end of the engagement when being detected is no longer important) to continually improve your knowledge of user behavior about password creation.

COMPLEX BUT PREDICTIBLE

Sometimes, you do not see how you could have predicted them

You do not find a way to improve your tactics :

```
MIMIKATZ 192.168.110.110 445 DC-01    \ASTAR\sophos ErlKs8_2zbM
MIMIKATZ 192.168.110.110 445 DC-01    \ASTAR\d.soria %%%FasciculePonnnèy
```

COMPLEX BUT PREDICTIBLE

Sometimes, you do not see how you could have predicted them

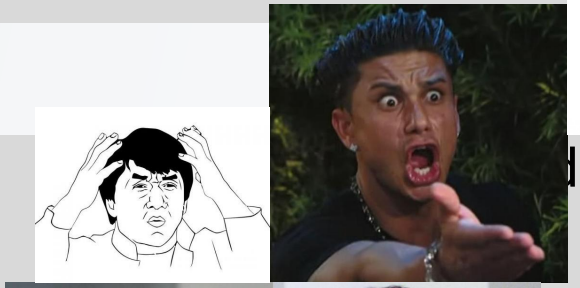
You do not find a way to improve your tactics :

```
MIMIKATZ 192.168.110.110 445 DC-01    \ASTAR\sophos ErlKs8_2z
MIMIKATZ 192.168.110.110 445 DC-01    \ASTAR\d.soria
```

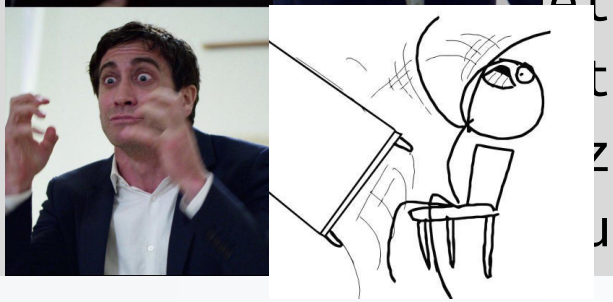


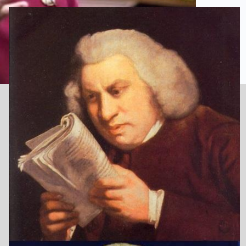
And sometimes you do :

```
MIMIKATZ 192.168.110.110 445 DC-01    dy j0ACHIM1
MIMIKATZ 192.168.110.110 445 DC-01    Thailande?
MIMIKATZ 192.168.110.110 445 DC-01    ert Enzo2004!
MIMIKATZ 192.168.110.110 445 DC-01    V334m@dm
MIMIKATZ 192.168.110.110 445 DC-01    as Pedr0.com
MIMIKATZ 192.168.110.110 445 DC-01    et Novembre2022!
MIMIKATZ 192.168.110.110 445 DC-01    t C@l!f0rn!@
MIMIKATZ 192.168.110.110 445 DC-01    z Woodstock1969.
MIMIKATZ 192.168.110.110 445 DC-01    u Wxcvbn12!
```

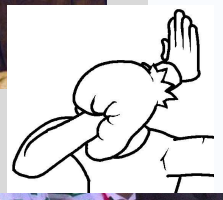












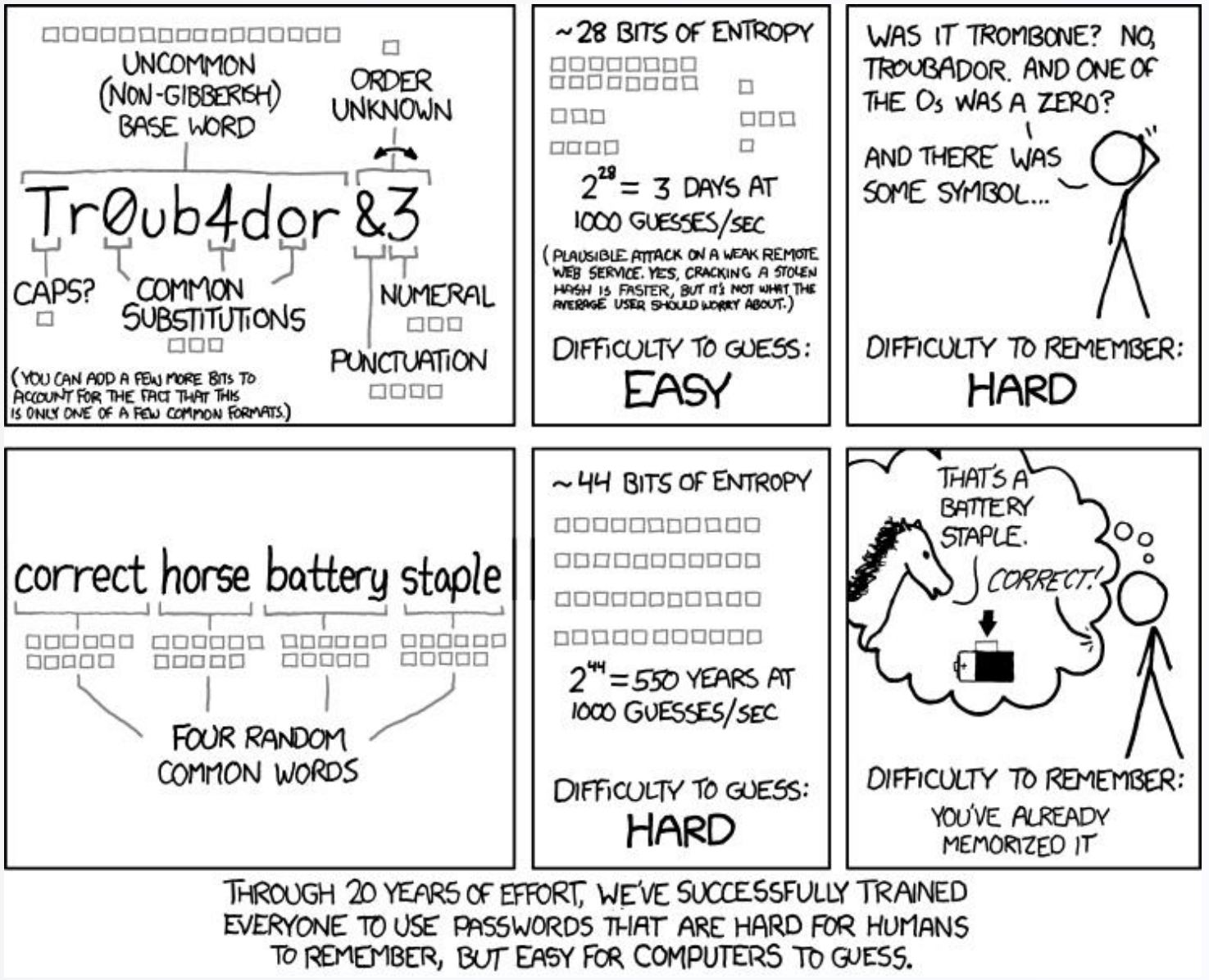


COMPLEX BUT PREDICTIBLE

We observe that people use a «root» (a pronounceable part) and embellishments (fancy ways to pimp the root) to construct a password.

Far from the idea in the mind of security experts who said «1 lowercase, 1 uppercase, 1 digit, 1 punctuation».

Find the root, find the common embellishments, and passwords are yours !



DEFEAT TR0UB4DOR&3

I used to take rsmangler to derivate passwords from likely roots :

```
$ cat words.txt
david
soria
lacanau
33
$ rsmangler --file words.txt > words_mangled.txt
$ wc words_mangled.txt
39924  39924 732204 word_mangled.txt
```

OK, it gives me: `david33! - $0r14d@v1d - lacanaudavid2009 - dsl31 - davidsoria33lacanau2017 - adminsoria33 - ...`

Not so bad

DEFEAT TR0UB4DOR&3

Not so bad but ... there’re some limits :

- If I feed it with : airbus, avion, a380, a350, 31, toulouse it will produce: a380aviona350airbus31toulouse2015 which is useless
- Beyond 4 ou 5 words, it becomes uncalculable
- I got david33!, David33, d@v!d33 but not D@vid33!

Time after time, I observed real passwords through mimikatz and I figured out they would not have been generated with rsmangler, even with the right roots.

To do this well, you have to use john rules or hashcat masks.

Effective ... **but not convenient**: I have to go deep in their *not-so-easy-to-remember* syntax each time I want to tune them (which actually mean each time).

DEFEAT TR0UB4DOR&3

I prefer python !

So, I coded **MikeThe Mangler.py** which precisely apply my strategies and combine them :

- Case variation (david David DAVID dAVID)
- Shortname caculation (nicolas -> nini, nic, nico, ncls)
- l33tspeak with 2 substitutions max (nicolas -> n1colas, n1col4s, nicol4s, ...)
- Derivation over the company name
- Derivation over the username
- Roots are combined 2x2 (DavidSoria, DavidLacanau, ~~DavidLacanauSoria~~)
- Suffixes based on birthdates (1208, 1994, ...)
- Tests common firstnames (for those who takes their children's firstname as password)
- ...

DEFEAT TR0UB4DOR&3

It was ok but, more and more, I figured out the embellishments were very specific on certain tactics and that I could avoid many useless attempts by selecting only those tactics relevant to a given context.

So, I split MikeTheMangler into 7 specific scripts :

- gen_company_wordlist.py (Airbus31!, Arbs31!, Air31!, ...)
- gen_dates_wordlist.py (Novembre2019! Summer2k20, ...)
- gen_names_wordlist.py (Zoé2012!, Paul0408, ...)
- gen_numbers_wordlist.py (from 0 to 99999999)
- gen_places_wordlist.py (T4nz4ni4, Toulouse31, ...)
- gen_user_wordlist.py (Fl0r3nt33$, FLO2018_, ...)
- gen_quickwin_wordlist.py (a versatile one for WiFi key, Web bruteforce, ...)

DEFEAT TR0UB4DOR&3

Each script generates wordlist files (between 30 MB and 30 GB) on the fly **in a few seconds/minutes.** So you only have to store the few kB of the scripts themselves. Code remains esay to read and tune on the fly (python). Here are some real passwords defeated by those scripts:

```
Lulu2509
Cho-2000
Emma1307!
T0ulouse31!
Rostov07
Avril2021$
Octavia1
aURELIA0205
Madinna972
28120107
July2019
```


ONE DAY YOU'LL MAKE DAD'S COMPANY LOSE MILLIONS EMMA1307

LOOT

WHO THE F**K IS THIS GUY ?

When you sucessfully break a hash, you look after the privileges of the compromised account.

In the begining I used a self-made rpcclient script which asks the DC for the status (enable/disable) and the groups of an account (or a list of accounts) and lists the compromised privileges :

```
$ sh getGroupsFromUsers.sh users_compromised.txt
--- domain admin
admdso
admqpo
--- backup operator
admdso
--- 27's club
kcobain
```

WHO THE F**K IS THIS GUY ?

Fair, but... it needs to call the DC each time.

And I don’t use predictible passwords attacks only. I also have a dedicated computer for pure bruteforce and leaked databases.

So, it happens that I finally crack a new hash through bruteforce, 2 days after the end of the pentest, and I cannot contact the DC anymore.

Is the account enabled or disabled ? Is it an admin or a simple user ?

You can tell the client that this account password is risky, but you cannot describe the impact, so you look dumb **(and you deserve it)**.

After twice experimenting this, I finally wrote a script fed with the file domain_users.grep from ldapdomaindump. It lets me know, **offline**, the status and groups of every user I ask for.

WHO THE F**K IS THIS GUY ?

I made a proper LesterTheLooter.py which takes the result of the command john --show and the file domain_users.grep :

```
$ python3 LesterTheLooter.py john_result.txt domain_user.grep
[*] Importing john result and domain info
[*] Exporting data to users_compromised.csv and group_compromised.csv
[+]  disabled  domain admins       admgba          LoveWine4Ever
[+]  enabled   likely admin        j.snow          knower0fTh1ngs
[+]  enabled   enterprise admins   l.finger        S4nsa<3
[+]  enabled   domain admins       adm.cla         JaimeJaime
[+]  disabled  account operators   t.lannister     K1ng'sHand
```

The outputs are easy-to-grep CSV files and a colored terminal summary of the accounts compromised (what you directly want to see during a pentest)

RESTITUTE

HERE ARE YOUR PASSWORDS

Giving back to a client the passwords you have compromised is always a good "Wow" effect at a lower cost

And if we got the hashdump of the DC, we can show some stats about the overall password habits. It's even better.

The pipal tool is well known for that purpose:

```
$ pipal passwords_cracked.txt
Total entries = 1360                One to six characters = 17 (1.25%)
Total unique entries = 121          One to eight characters = 1297 (95.37'%)
                                    More than eight characters = 63 (4.63%)
Top 10 passwords
P@ssword = 1236 (90.88%)            First capital last symbol = 16 (1.18%)
1234Abcd= 3 (0.22%)                 First capital last number = 106 (7.79%)
...                                 ...
```

HERE ARE YOUR PASSWORDS

Except that, rendering stats directly in ASCII art, without formatting, is not really what you expect as a client.

Formatting them is boring and redundant.

Moreover, some stats from pipal are useless while other we would like to have are lacking.

Thus, I add a dedicated stats CSV file as an output of LesterTheLooter.py :

- user_compromised.csv: for each user compromised shows its groups
- group_compromised.csv: for each group shows the members compromised
- lestat.csv: many statistics, including a heuristic evaluation of the robustness and the reason for weakness (too short, too similar to login or company name, too much known, ...). You can also manually provide your reasons if you wish.

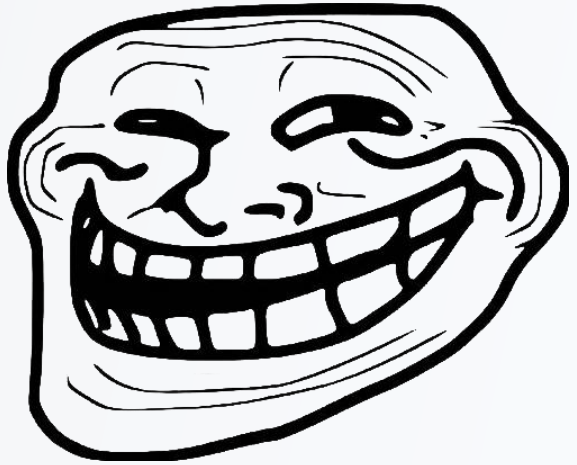
HERE ARE YOUR PASSWORDS

So you just have to import those 3 CSV files into 3 dedicated sheets, in one Excel file, to render a clean and exploitable deliverable to the client.

HERE ARE YOUR PASSWORDS

So you just have to import those 3 CSV files into 3 dedicated sheets, in one Excel file, to render a clean and exploitable deliverable to the client.

TOO LONG



Then I added a --xlsx option into LesterTheLooter.py to automatically generate the Excel file with all the data imported, stick the first line, all the formatting, color highlight, ...

(xlsxwriter python module is so easy to use, it took me only a day)

HERE ARE YOUR PASSWORDS

	A	B	C	D	E	F	G	H	I	J
1	Username	Password	Status	Last logon	Last change	# group	Groups	Sensitivity	Robustne	Reason
38			enabled	12/18/20 16:00:46	11/12/20 07:36:29	18		unknown	days	common
39			enabled	03/13/17 07:06:37	03/27/17 07:01:33	2		unknown	minutes	top1000
40			disabled		04/07/16 11:30:17	2	domain admins, domain users	domain admins	hours	top1M
41			enabled	12/03/20 10:39:36	11/17/20 07:01:33	13		unknown	days	common
42			enabled	12/16/20 14:42:41	02/28/19 12:32:28	6		unknown	minutes	login derivation
43			enabled		06/06/16 07:34:40	2	, domain users	unknown	hours	top1M
44			enabled		06/07/16 13:14:41	2		unknown	hours	top1M
45			enabled	12/18/20 06:26:53	06/16/16 11:19:11	9		unknown	minutes	login derivation
46			enabled		11/23/16 12:08:03	2	, domain users	unknown	hours	top1M
47			enabled	12/28/20 15:28:32	12/07/20 06:46:51	25		likely admin	days	common
48			enabled		12/07/16 11:56:06	2	, domain users	unknown	minutes	top1000
49			enabled		01/23/17 15:13:28	2	, domain users	unknown	hours	top1M
50			enabled		01/25/17 09:23:41	2	, domain users	unknown	hours	top1M
51			enabled	11/18/20 08:45:03	02/19/20 10:51:34	8		enterprise admins	days	common
52			enabled	12/23/20 07:28:16	11/23/20 07:43:59	12		likely admin	days	common
53			disabled	11/06/17 14:57:26	11/06/17 14:47:56	5		unknown	days	common
54			enabled	12/24/20 16:29:46	07/24/17 13:54:19	3	sg-citrixmcs-office, remote desktop users, domain users	unknown	days	common
55			enabled	07/27/17 14:46:07	07/27/17 09:21:55	2	remote desktop users, domain users	unknown	days	common

HERE ARE YOUR PASSWORDS

users_compromised.csv provides:

Name	Passwd	Status	lastlogon	lastchange	#groups	groups	sensitive	robustness	Reason
admdso	Astar2021	enabled	2021-04-06	2021-01-03	12	domain admins, ...	admin	minutes	company
kcobain	N!rv@na	disabled	1994-04-05	1994-01-12	2	27 club, ...	unknown	hours	common

groups_compromised.csv provides:

Name	members cracked	Sensitive	robustness	enabled members	disabled members
admins du domaine	3	admin	seconds	admdso, admqpo	adminbackup
club des 27	1	unknown	years		kcobain

HERE ARE YOUR PASSWORDS

lestat.csv provides all the relevant tables:

- Number of cracked accounts (enabled and disabled)
- Number of admin accounts compromised
- Distribution of the cracked passwords by length, complexity, charsets, robustness, reason of weakness, ...

You just have to produce the charts from those tables in Excel.

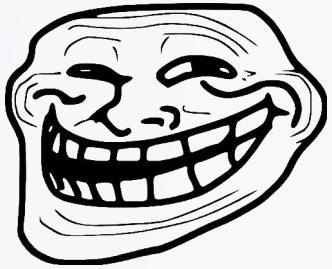
HERE ARE YOUR PASSWORDS

lestat.csv provides all the relevant tables:

- Number of cracked accounts (enabled and disabled)
- Number of admin accounts compromised
- Distribution of the cracked passwords by length, complexity, charsets, robustness, reason of weakness, ...

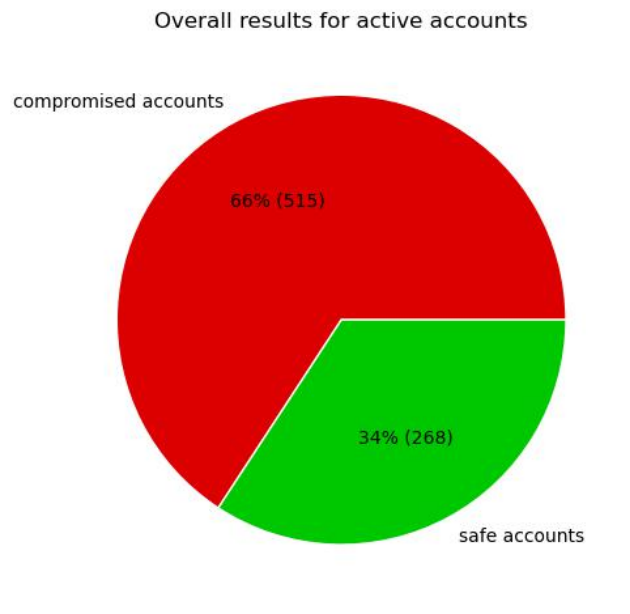
You just have to produce the charts from those tables in Excel.

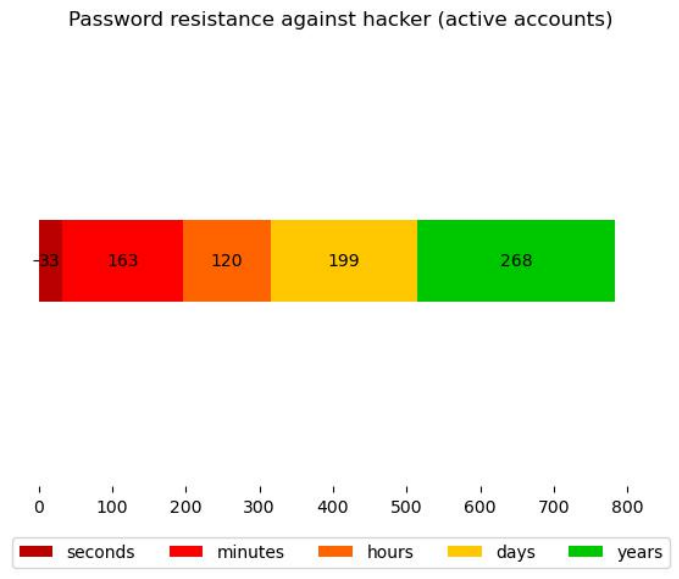
TOO LONG

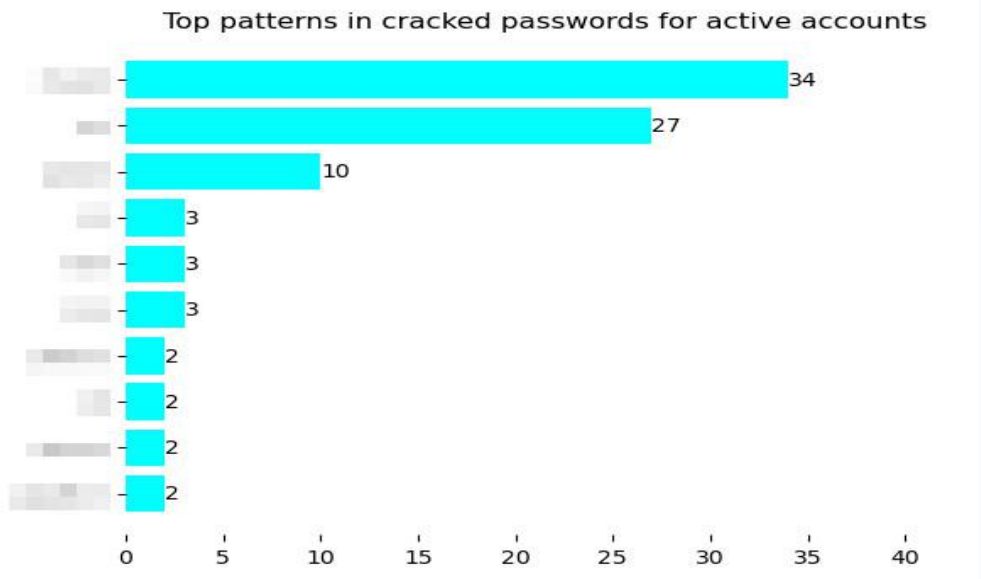


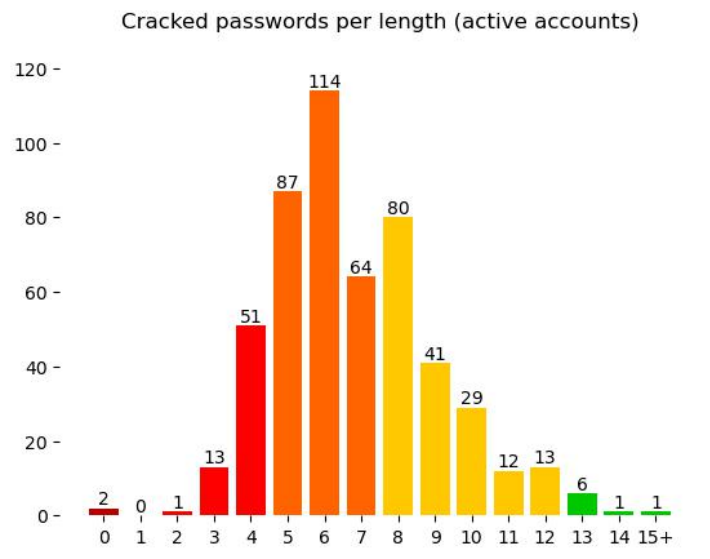
Then, I coded GregTheGrapher.py. Feeded with lestat.csv file, it produces all the usefull charts (a series with all accounts, another with the active ones only)

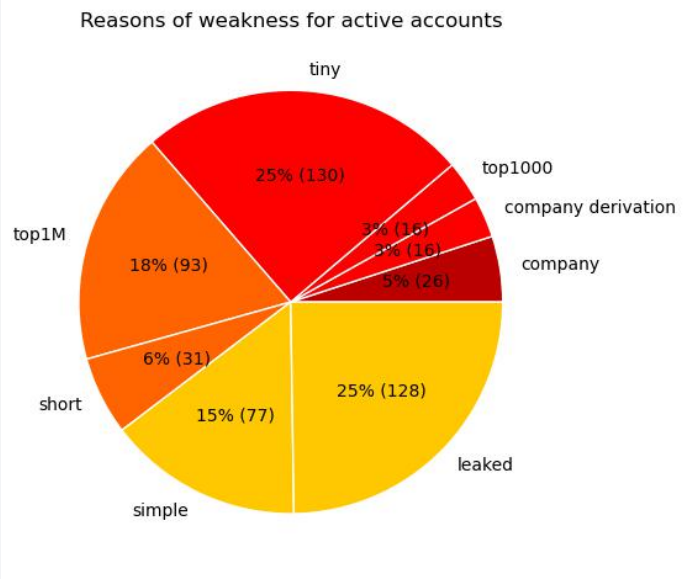
HERE ARE YOUR PASSWORDS

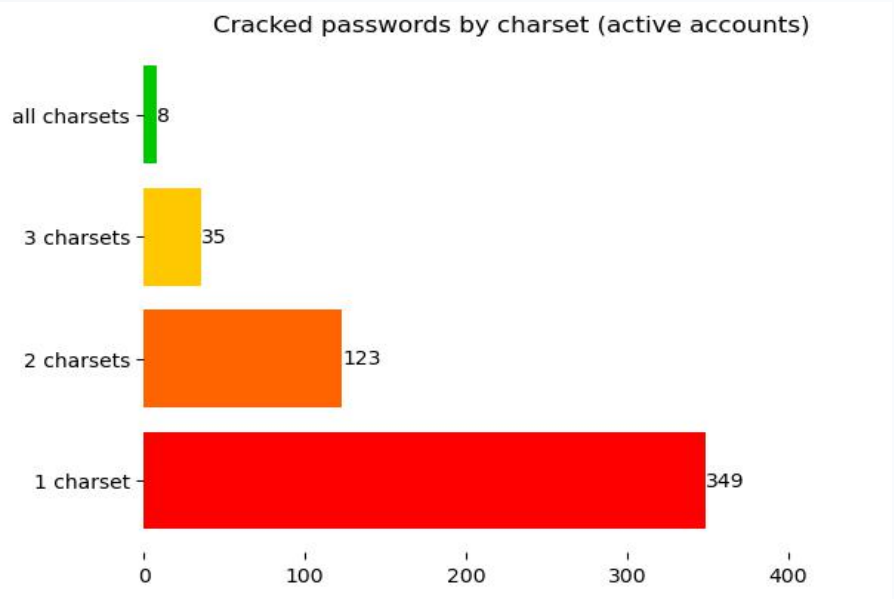












ON THE SHELF

PASSWORD ROBUSTNESS AS A SERVICE

This set of tools was designed as a «time saver» for pentesters

BUT, I say to all my clients *«You should test the robustness of your users passwords every year»* **AND** there is nothing on the shelf to do it.

I thought that, with some effort, I could make my tools relatively plug-and-play.

I coded RobbTheRobber.py. Fed with a domain admin account, it execute the ldapdomaindump and the secretsdump :

```
$ ./RobbTheRobber.py --user admdso --password poulpe --domain astar.lcl
```

It produces 2 files : USERS_INFO.txt (LDAP dump result) and DOMAIN_HASHES.txt (secretsdump result)

PASSWORD ROBUSTNESS AS A SERVICE

But that wasn't enough, because it's a bit light to just tell them "then you crack the hashes with john-the-ripper or hashcat, export the result and we'll get together later".

So, ... I coded a JohnTheRipper

PASSWORD ROBUSTNESS AS A SERVICE

But that wasn't enough, because it's a bit light to just tell them "then you crack the hashes with john-the-ripper or hashcat, export the result and we'll get together later".

So, ... I coded a JohnTheRipper.**py**

Yeah it’s stupid from a performance point of view.

But, I rewrote the gen_X scripts so they can be called in a standalone mode **OR** be imported and used on-the-fly (fileless) with a hash function.

It’s not pure bruteforce. Only dictionnary attacks. It takes 30 minutes.

Not perfect, but I closed the cycle for the clients who need a full automated process.

RECAP

LAZY AT WILL

For a non specialist, it is «quite easy» to use :

Only 3 inputs and 4 commands:

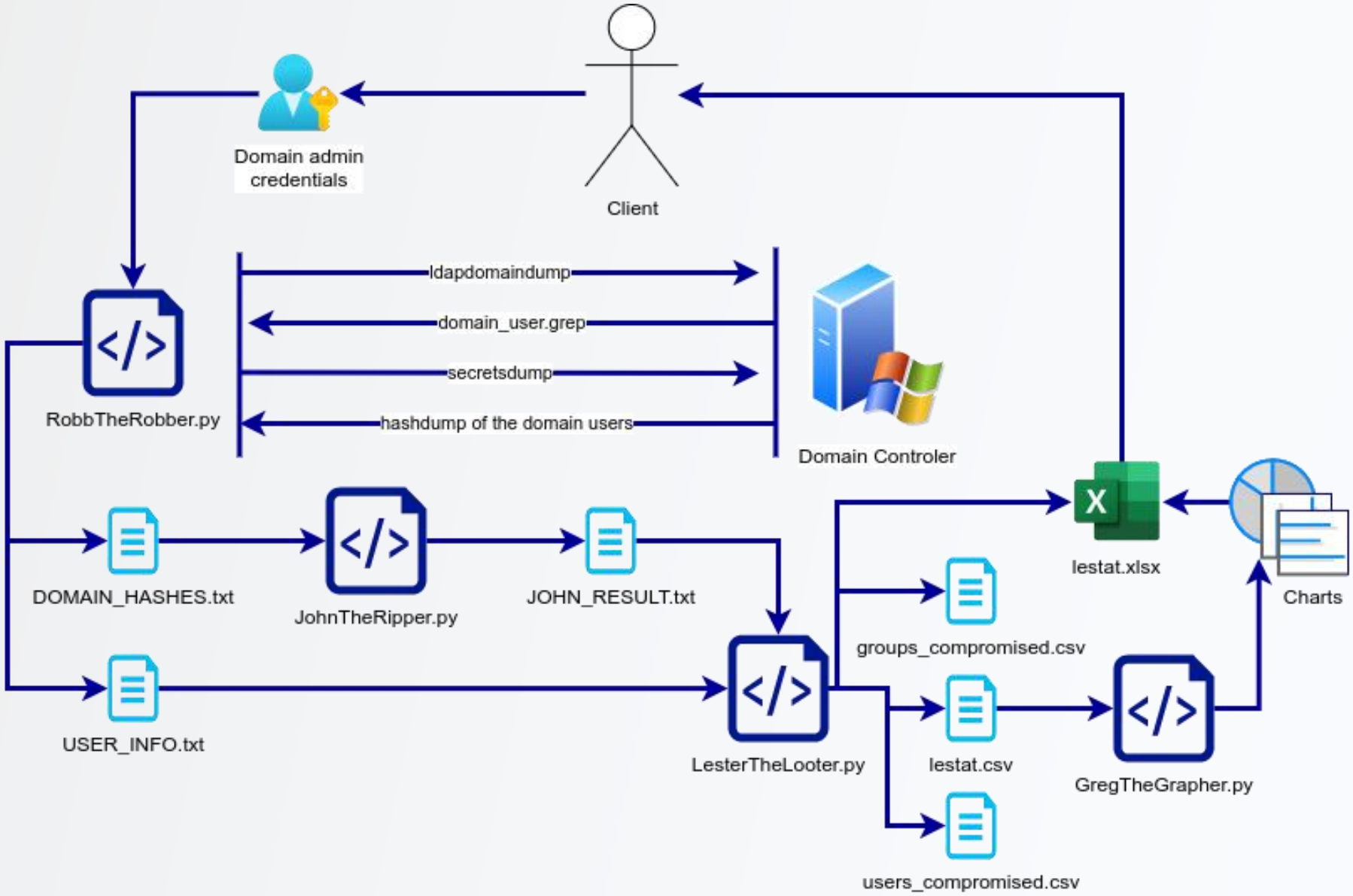
```
$ ./RobbTheRobber.py --user admdso --password poulpe --domain astar.org
$ ./JohnTheRipper.py DOMAIN_HASHES.txt
$ ./LesterTheLooter.py --xlsx JOHN_RESULT.txt USERS_INFO.txt
$ ./GregTheGrapher.py lestat.csv
```

Every tool can also be used in a standalone mode.

The dependencies are :

- ldapdomaindump
- impacket
- click

LAZY AT WILL (OK MAYBE NOT)



ROADMAP

- Make a one liner instead of 4 commands
- Use Julia instead of Python for performance (no MD4 library yet)
- Manage the case where a user has a password with ‘:’ inside
- Improve recognition of admin groups
- Use `domain_users.json` instead of `domain_users.grep`
- Incorporate all the gen_X scripts in JonhTheRipper.py
- Generate the charts directly in Excel
- Feed GregTheGrapher with the excel file to remove the need of intermediate CSV files
- Test it on Windows

LAZY GUY MADES LAZY NAME

As the tool is dedicated to produce the metrics about passwords, I called it «themetrics» but in French : lestat



THANKS FOR YOUR ATTENTION

URL of the project : https://github.com/astar-security/Lestat